

R intermédiaire

Atelier intensif

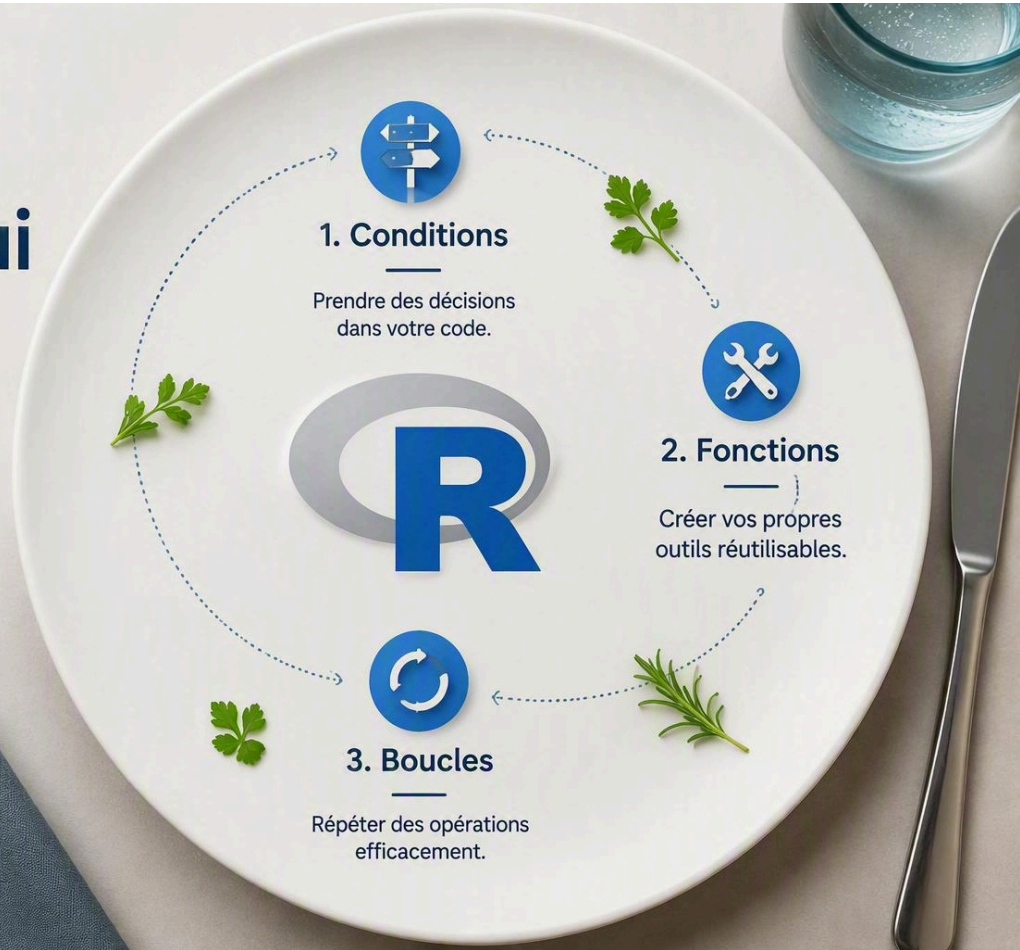
Guillaume Boucher

INSPQ

Le 10 septembre 2026

Au menu aujourd'hui

3 ingrédients essentiels
pour cuisiner vos données
avec **R** !



L'objectif est de découvrir *les ingrédients* essentiels pour automatiser ses analyses en R.

Conditions

Qu'est-ce qu'une condition

Une condition permet à R de prendre une décision selon qu'une règle est vraie (TRUE) ou fausse (FALSE).

On utilise le mot-clé conditionnel `if` pour créer la condition (si) :

```
if (condition) {  
  action si TRUE  
}
```

On ajoute le mot-clé conditionnel `else` (sinon) pour ajouter une condition par défaut :

```
if (condition) {  
  action si TRUE  
} else {  
  action si FALSE  
}
```

Les conditions sont entre parenthèses `()` et les actions entre les accolades `{}`.

Exemple

Si j'ai 65 ans ou plus
 Je suis un aîné
Sinon
 Je ne suis pas un aîné.

```
age <- 45
if (age >= 65) {
  print("Je suis un aîné")
} else {
  print("Je ne suis pas un aîné")
}
```

```
#> [1] "Je ne suis pas un aîné"
```

Plusieurs conditions

Lorsqu'il y a plusieurs conditions, la deuxième condition et les suivantes se font avec les mots-clés `else if` (sinon si).

Suis-je un aîné, un adulte ou un mineur ?

```
age <- 45
if (age >= 65) {
  print("Aîné")
} else if (age >= 18) { # < 65 géré par le premier si
  print("Adulte")
} else if (age < 18 & age >= 0) {
  print("Mineur")
} else {
  print(NA) # utile pour trouver des erreur (âge négatif)
}
```

```
#> [1] "Adulte"
```

Conditions dans une condition

Les conditions peuvent s'imbriquer les unes dans les autres.

```
if (note >= 60) {  
    if (note >= 90) {  
        print("A")  
    } else {  
        print("Réussite")  
    }  
} else {  
    if (note >= 50) {  
        print("Échec de peu")  
    } else {  
        print("Échec")  
    }  
}
```

Condition sur un vecteur

`if` est utilisé pour prendre une décision dans un programme.
On utilise `ifelse()` pour appliquer une règle à toutes les valeurs d'un vecteur ou d'une colonne.

```
age <- c(72, 54, 22, 65, 18)
ifelse(
  test = age >= 65, # condition
  yes = "Aîné",    # valeur si vrai
  no  = "Adulte"   # valeur si faux / par défaut
)
```

```
#> [1] "Aîné" "Adulte" "Adulte" "Aîné" "Adulte"
```

S'il y a plusieurs conditions :

```
age <- c(72, 54, 22, 65, 7)
ifelse(
  age >= 65, # condition 1
  "Aîné", # valeur si condition 1 est vraie
  ifelse(age > 18, # condition 2
    "Adulte", # valeur si condition 2 est vraie
    "Mineur") # valeur par défaut
)
```

```
#> [1] "Aîné" "Adulte" "Adulte" "Aîné" "Mineur"
```

ifelse - solutions alternatives

Selon la documentation, les alternatives ci-dessous seraient plus performantes et corrigeraient certaines incohérences du `ifelse()` de base.

package dplyr

```
age <- c(72, 54, 22, 65, 18)
dplyr::if_else(age >= 65, "Aîné", "Adulte")
```

```
#> [1] "Aîné" "Adulte" "Adulte" "Aîné" "Adulte"
```

package vctrs

```
age <- c(72, 54, 22, 65, 18)
vctrs::vec_if_else(age >= 65, "Aîné", "Adulte")
```

```
#> [1] "Aîné" "Adulte" "Adulte" "Aîné" "Adulte"
```

Fonctions

Problème ?

Voyez-vous un problème ici ?

```
age <- 88
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 44
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 46
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 36
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 17
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 5
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

Utilité d'une fonction

Une fonction permet d'enregistrer une série d'instructions sous un nom afin de pouvoir les réutiliser.

```
groupe_age <- function(age) {  
  if (age >= 65) {  
    return("Aîné")  
  } else {  
    return("Non aîné")  
  }  
}
```

```
class(groupe_age)
```

```
#> [1] "function"
```

```
groupe_age(88)
```

```
#> [1] "Aîné"
```

```
groupe_age(36)
```

```
#> [1] "Non aîné"
```

```
grp_age <- groupe_age(44)  
grp_age
```

```
#> [1] "Non aîné"
```

Structure d'une fonction

Nom de la fonction

On commence par inscrire le nom de la fonction qu'on veut créer :
`groupe_age`.

Créer la fonction

On crée un objet fonction avec le mot-clé `function`.

```
groupe_age <- function()
```

Arguments

On inscrit les arguments à l'intérieur des parenthèses.

```
groupe_age <- function(argument1, argument2, ...)
```

Corps de la fonction

On insère le code que la fonction doit exécuter à l'intérieur des accolades :

```
groupe_age <- function(argument1, argument2) {  
  code à exécuter  
}
```

Exemple d'une fonction sans arguments :

```
msg_bienvenue <- function() { # fonction sans arguments  
  paste("Aujourd'hui nous sommes le", Sys.Date())  
}  
msg_bienvenue()
```

Position et valeur des arguments

Prenons par exemple la fonction `median` :

```
args(median)
```

```
#> function (x, na.rm = FALSE, ...)  
#> NULL
```

- **x** : *an object for which a method has been defined, or a numeric vector containing the values whose median is to be computed.*
- **na.rm = FALSE** : *a logical value indicating whether NA values should be stripped before the computation proceeds.*
- **...** : *potentially further arguments for methods; not used in the default method.*

Le premier argument `x` n'a pas de valeur par défaut, il est donc **nécessaire** sinon il y aura une erreur.

```
median()
```

```
#> Error in `median.default()`:  
#> ! l'argument "x" est manquant, avec aucune valeur par défaut
```

L'argument `na.rm` a la valeur `FALSE` par défaut alors il n'est pas nécessaire de l'inclure si la valeur par défaut est celle qu'on veut utiliser.

```
median(x = 1:4)
```

```
#> [1] 2.5
```

```
median(x = 1:4, na.rm = FALSE)
```

```
#> [1] 2.5
```

Il n'est pas nécessaire de nommer les arguments lorsque l'ordre est respecté.

```
median(1:4, FALSE)
```

```
#> [1] 2.5
```

Au contraire, il est nécessaire de les nommer lorsque l'ordre n'est pas respecté.

```
median(na.rm = FALSE, x = 1:4)
```

```
#> [1] 2.5
```

Construire une fonction

Voici un exemple de comment construire une fonction à partir de son code.
On veut calculer l'indice de masse corporel.

```
poids <- 80 # kg  
taille <- 1.75 # mètres  
poids / taille ^ 2
```

```
#> [1] 26.12245
```

On décide que le nom de la fonction est `calcul_imc` :

```
calcul_imc <- function() {  
  
}
```

Quels seraient les arguments ?

```
poids <- 80  
taille <- 1.75  
poids / taille ^ 2
```

Les arguments sont le poids et la taille

```
calcul_imc <- function(poids, taille) {  
  
}
```

Il reste à inclure le code à exécuter

```
calcul_imc <- function(poids, taille) {  
  poids / taille ^ 2  
}
```

Si mon poids est de 80 kg et ma taille est de 1,75 mètres, mon IMC serait de

```
calcul_imc(poids = 80, taille = 1.75)
```

```
#> [1] 26.12245
```

Retour d'une fonction

R accepte deux styles pour retourner la valeur d'une fonction :

1. Retour implicite : la dernière ligne de code.

```
calcul_imc <- function(poids, taille) {  
  poids / taille ^ 2  
}
```

1. Retour explicite : utiliser `return()`.

```
calcul_imc <- function(poids, taille) {  
  return(poids / taille ^ 2)  
}  
  
calcul_imc <- function(poids, taille) {  
  imc_val <- poids / taille ^ 2  
  return(imc_val)  
}
```

Boucles

Pourquoi utiliser une boucle

Une boucle permet de répéter un même bloc de code plusieurs fois de suite.

Analogie simple :

- Sans boucle, on dit : *Bonjour A, bonjour B, bonjour C, ...*
- Avec une boucle, on dit : *Pour chaque personne dans la liste, dit bonjour.*

For

S'utilise quand on sait exactement combien de fois on veut répéter l'action.

```
for (i in 1:3) {  
  print(i)  
}
```

```
#> [1] 1  
#> [1] 2  
#> [1] 3
```

```
for (lettre in c("a","b","c")) {  
  print(lettre)  
}
```

```
#> [1] "a"  
#> [1] "b"  
#> [1] "c"
```

Reviens à exécuter manuellement :

```
i <- 1  
print(i)  
i <- 2  
print(i)  
i <- 3  
print(i)
```

```
lettre <- "a"  
print(lettre)  
lettre <- "b"  
print(lettre)  
lettre <- "c"  
print(lettre)
```

While

Utile quand on ne sait pas combien de tours il faut faire, mais on sait **quand** on doit arrêter.

Exemple où on ne connaît pas la valeur de x , mais on sait que si x est plus grand que 5 on doit arrêter.

```
while (x <= 5) {  
  print(x)      # 2, 3, 4, 5  
  x <- x + 1    # 3, 4, 5, 6 est plus grand que 5 -> fin boucle  
}
```

```
#> [1] 2  
#> [1] 3  
#> [1] 4  
#> [1] 5
```

lapply

C'est une fonction en R qui répète une même action sur chaque élément et renvoie le résultat sous la forme d'une liste.

```
args(lapply)
```

```
#> function (X, FUN, ...)  
#> NULL
```

- *X : a vector (atomic or list) or an expression object. Other objects (including classed objects) will be coerced by base::as.list.*
- *FUN : the function to be applied to each element of X: see 'Details'. In the case of functions like +, %%, the function name must be backquoted or quoted.**

Exemple disponible dans la documentation :

```
x <- list(a = 1:10, beta = exp(-3:3),  
          logic = c(TRUE, FALSE, FALSE, TRUE))  
lapply(x, mean)
```

```
#> $a  
#> [1] 5.5  
#>  
#> $beta  
#> [1] 4.535125  
#>  
#> $logic  
#> [1] 0.5
```

```
# Exemple de ifelse dans les diapos précédentes
age <- c(72, 54, 22, 65, 18)
ifelse(
  test = age >= 65, # condition
  yes = "Aîné",    # valeur si vrai
  no  = "Adulte"   # valeur si faux / par défaut
)
```

```
lapply(age, function(x) {
  if (x >= 65) {
    return("Aîné")
  } else {
    return("Adulte")
  }
})
```

```
#> [[1]]
#> [1] "Aîné"
#>
#> [[2]]
#> [1] "Adulte"
#>
#> [[3]]
#> [1] "Adulte"
#>
#> [[4]]
#> [1] "Aîné"
#>
```

Exemple sur un data.frame :

```
lapply(iris[, 1:3], mean)
```

```
#> $Sepal.Length  
#> [1] 5.843333  
#>  
#> $Sepal.Width  
#> [1] 3.057333  
#>  
#> $Petal.Length  
#> [1] 3.758
```

Comparativement à une boucle for :

```
for (col in names(iris[, 1:3])) {  
  print(mean(iris[[col]]))  
}
```

```
#> [1] 5.843333  
#> [1] 3.057333  
#> [1] 3.758
```

sapply

Le l dans lapply est pour list alors que le s dans sapply est pour simplifié

```
sapply(iris[, 1:3], mean)
```

```
#> Sepal.Length Sepal.Width Petal.Length  
#>      5.843333      3.057333      3.758000
```

```
sapply(iris[, 1:3], mean, simplify = FALSE) # Équivalent à lapply
```

```
#> $Sepal.Length  
#> [1] 5.843333  
#>  
#> $Sepal.Width  
#> [1] 3.057333  
#>  
#> $Petal.Length  
#> [1] 3.758
```

À retenir

- Les **conditions** permettent de prendre des décisions.
- Les **fonctions** permettent de réutiliser du code.
- Les **boucles** permettent de répéter un traitement.
- Ensemble, elles permettent d'écrire du code plus simple, robuste et maintenable.

Question à se poser :

Est-ce que je suis en train de répéter quelque chose qui pourrait être automatisé ?